LAB-03- I/O REDIRECTION

- **Learning Outcomes**
  - I/O Redirection on Terminal using & operator.
  - File Handling in C++
  - I/O Redirection using dup, dup2 system call.

Connection of an Opened File

File Management in Linux

Following are the four key system calls for performing file I/O (programming languages and software packages typically employ these calls indirectly via I/O libraries):

- **fd = open(pathname, flags, mode)** opens the file identified by pathname, returning a file descriptor used to refer to the open file in subsequent calls. If the file doesn't exist, open() may create it, depending on the settings of the flags bit. The flags argument also specifies whether the file is to be opened for reading, writing, orboth. The mode argument specifies the permissions to be placed onthe file if it is created by this call. If the open() call is not being used to create a file, this argument is ignored and can be omitted
- **numread = read(fd, buffer, count)** reads at most count bytes from the open file referred to by fd and stores them in buffer. The read() call returns the number of bytes actually read. On eof, read() returns 0.
- **numwritten = write(fd, buffer, count)** writes up to count bytes frombuffer to the open file referred to by fd. The write() call returns the number of bytes actually written,

which may be less than count

- **status = close(fd)** is called after all I/O has been completed, in order to release the file descriptor fd and its associated kernel resources.

File Descriptor to File Contents

- Each process in UNIX has an associated PPFDT, whose sizeis equal to the number of files that a process can open simultaneously
- File descriptor is an integer returned by **open()** system call,

  and is used as an index in the PPFDT
- File descriptor is used in **read()**, **write()** and **close()** systemcall
- Kernel uses this descriptor to index the PPFDT, which contain a pointer to another table called **System Wide FileTable**
- In the **System Wide File Table,** other than some information there is another pointer to a table called **InodeTable**
- The inode table contains a unique Inode to every unique fileon disk.

Standard Descriptors in UNIX / Linux

- Three files are automatically opened for every process toread its input from and to send its output and error messages to.

- These files are called standard files:
  - **0 – Standard Input (stdin).** Default input to a program is from the user terminal (keyboard), if no filename is given.
  - **1 – Standard Output (stdout).** A simple program's output normally goes to the user terminal (monitor), ifno file name is given.
  - **2 – Standard Error (stderr).** Default output of errormessages from a program normally goes to the user terminal, if no file name is given.
- These numbers are called File Descriptors – System callsuse them to refer to files.

Examples – File Handling

Example 1:

```
#include <unistd.h>  // For read, write
#include <fcntl.h>   // For open

int main() {
   char buff[256];
   read(0, buff, 255);
   write(1, buff, 255);
```

```
    return 0;
}
```

Example 2:

```
#include <iostream>
#include <unistd.h>  // For read and write

int main() {
    char buff[256];
    while (true) {
        int n = read(0, buff, 255);  // Reading from stdin
        if (n <= 0) break;           // Exit loop if no input or error
        write(1, buff, n);           // Writing to stdout
    }
    return 0;
}
```

Example 3:

```
#include <iostream>   // For C++ I/O
#include <unistd.h>   // For read, write
#include <fcntl.h>    // For open
#include <cstdlib>    // For exit
int main() {
```

```
  int n;
  char buff[1024];

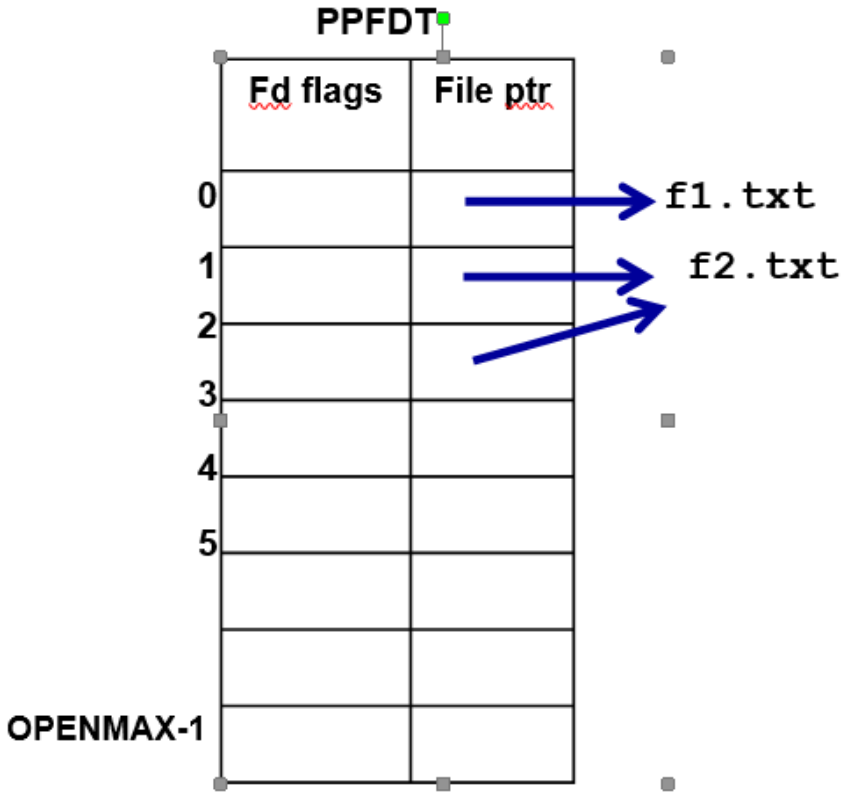
  // Open the file with appropriate flags and permissions
  int fd = open("file.txt", O_CREAT | O_TRUNC | O_RDWR, 0666);
  if (fd < 0) {
     std::cerr << "Error opening file.\n";
     return 1;
  }

  // Infinite loop to read from stdin and write to the file
  while (true) {
     n = read(0, buff, 1023);  // Reading from stdin (fd 0)
     if (n <= 0) {
        std::cerr << "Error in reading from stdin.\n";
        exit(-n);  // Handle error or EOF
     }
     write(fd, buff, n);  // Writing to file
  }

  close(fd);  // Close the file
  return 0;
}
```

Duplicating a File Descriptor

$ cat 0< f1.txt 1> f2.txt 2>&1



Tasks

Differentiate between following two commands (if file2 do not exist)

```
$ cat < file1 > file2
$ cp file1 file2
```

- Differentiate between following two commands (if lab1 do not exist)

```
$ cat lab1.txt 1> output.txt 2> error.txt
$ cat 0< lab1.txt 1> output.txt 2> error.txt
```

dup() System call

```
int dup(int oldfd);
```

- We know that `dup()` call guarantees that the new descriptor returned is the lowest unused file descriptor
- If we run the following LOCs, the `open` call will return 3, the `dup` call will return the lowest unused descriptor which will be zero. So finally descriptor zero points to the opened file instead of `stdin`

```
fd = open(...);

close(0);

newfd = dup(fd);
```

- To make the above code simpler, and to ensure we always get the file descriptor we want, we can use **`dup2()`**

`dup2()` System call

```
int dup2(int oldfd, int newfd);
```

- The `dup2()` system call makes a duplicate of the file descriptor given in `oldfd` using the descriptor number supplied in `newfd`
- If the file descriptor specified in `newfd` is already open, `dup2()` closes it first
- We can simplify the preceding calls to `close(0)` and `dup(fd)` on previous slide to the following:

  ```
  dup2(fd, 0);
  ```

- A successful `dup2()` call returns the number of the duplicate descriptor (i.e., the value passed in `newfd`)
- If `oldfd` is a valid file descriptor, and `oldfd` and `newfd` have the same value, then `dup2()` does nothing—`newfd` is not closed, and `dup2()` returns the `newfd`

`dup3()` System call

```
int dup3(int oldfd, int newfd, int flags);
```

- The `dup3()` system call performs the same task as `dup2()`, but adds an additional argument, flags, that is a bit mask that modifies the behavior of the system call
- At the time of this writing, `dup3()` supports one flag, O_CLOEXEC, which causes the kernel to enable the close-on-exec flag (FD_CLOEXEC) for the new file descriptor
- The `dup3()` system call is new in Linux 2.6.27, and is Linux-specific

Examples: Input Redirection

Method 1: `close-open (stdinredir1.c)`

```
close(0);
fd = open("/etc/passwd", O_RDONLY);
```

Method 2: open-close-dup-close (stdinredir2.c)

```
fd = open("/etc/passwd", O_RDONLY);
close(0);
newfd = dup(fd);
close(fd);
```

Method 3: open-dup2-close (stdinredir3.c)

```
fd = open("/etc/passwd", O_RDONLY);
newfd = dup2(fd, 0);
close(fd);
```

In Lab Task

- Write a program using dup() system call that duplicated file descriptor 0,1,2,3 to a single file.
- Do the above task using dup2().